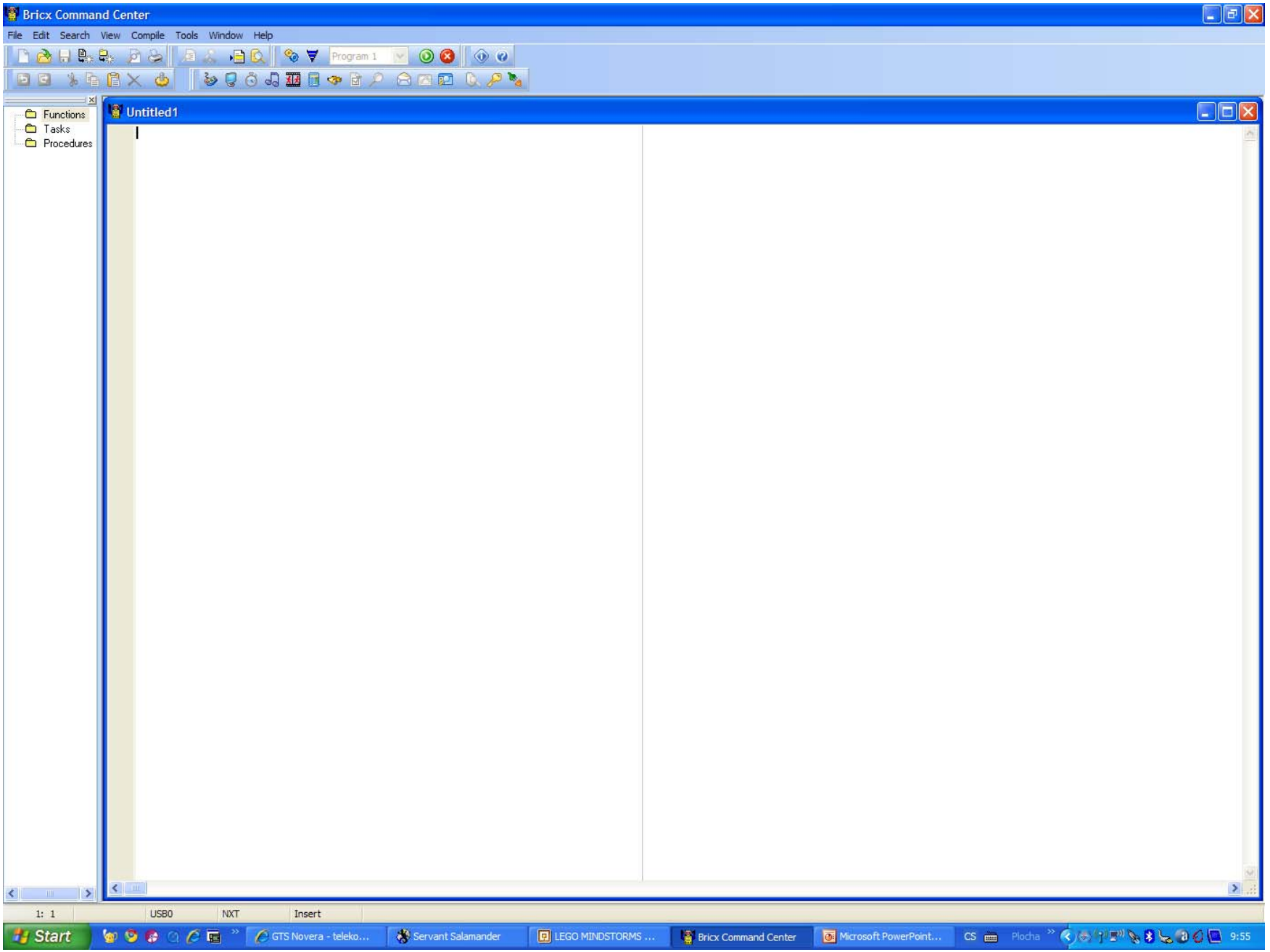
Programming LEGO NXT Robots using NXC

This text programming language derived from C language is bended together with IDE BricxCC on standard firmware LEGO Mindstorms. This can be very convenient for those, who want to program both in NXC and NXT-G, because it is not necessary to upload a new firmware every time you need to change the programming language. Working with the language which abbreviation stands for „Not eXactly C“ is very convenient and the programmer experienced at C language will get used to this language very soon. The next advantage is that this language is freeware. On the other hand the disadvantage would probably be the difficult debugging of the programs. In contrast to NXT-G this is pure text programming without the graphical blocks..

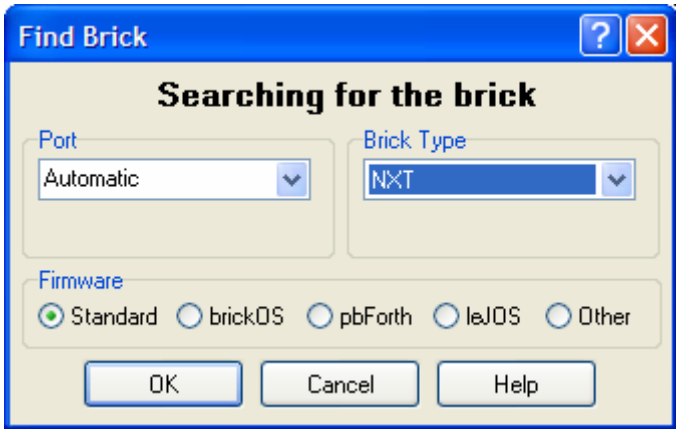
Where to get it?:

http://sourceforge.net/projects/bricxcc/

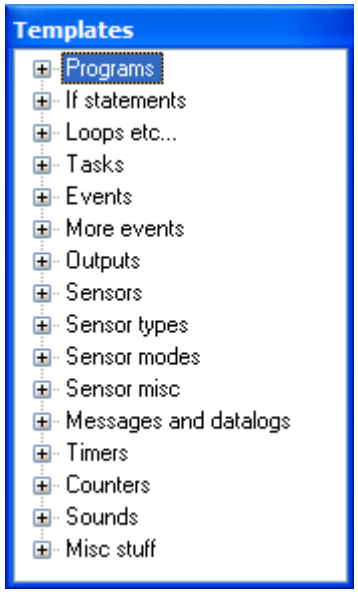
There is a tutorial for installation offered.



Window Bricx Command Center

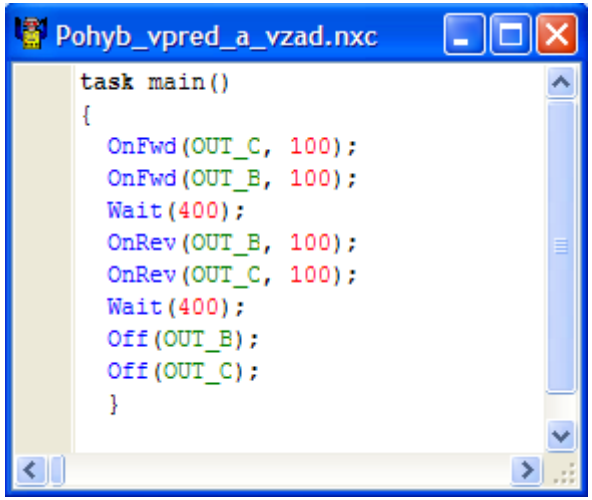


Starting Bricx Command Center. We write our programs using Bricx Command Center. Start it by double clicking on the icon BricxCC. (I assume you already installed BricxCC. If not, download it from the web site (see the preface), and install it in any directory you like. The program will ask you where to locate the robot. Switch the robot on and press OK. The program will (most likely) automatically find the robot.



If you press button F9, you can see the window Templates, which help you with the work.

The interface looks like a standard text editor, with the usual menu, and buttons to open and save files, print files, edit files, etc. But there are also some special menus for compiling and downloading programs to the robot and for getting information from the robot. You can ignore these for the moment. We are going to write a new program. So press the **New File** button to create a new, empty window. Writing the program:



It might look a bit complicated at first, so let us analyze it. Programs in NXC consist of tasks. Our program has just one task, named main. Each program needs to have a task called main which is the one that will be executed by the robot. A task consists of a number of commands, also called statements. There are brackets around the statements such that it is clear that they all belong to this task. Each statement ends with a semicolon. In this way it is clear where a statement ends and where the next statement begins.

Our program has six statements. Let us look at them one at the time:
OnFwd(OUT_C, 75) - This statement tells the robot to start output C, that is, the motor connected to the output labeled C on the NXT, to move forwards. The number following sets the speed of the motor to 75% of maximum speed.

OnFwd(OUT_B, 75) - Same statement but now we start motor B. After these two statements, both motors are running, and the robot moves forwards.

Wait(400) - Now it is time to wait for a while. This statement tells us to wait for 4 seconds. The argument, that is, the number between the parentheses, gives the number of 1/100 of a second: so you can very precisely tell the program how long to wait. For 4 seconds, the program is sleeping and the robot continues to move forwards.

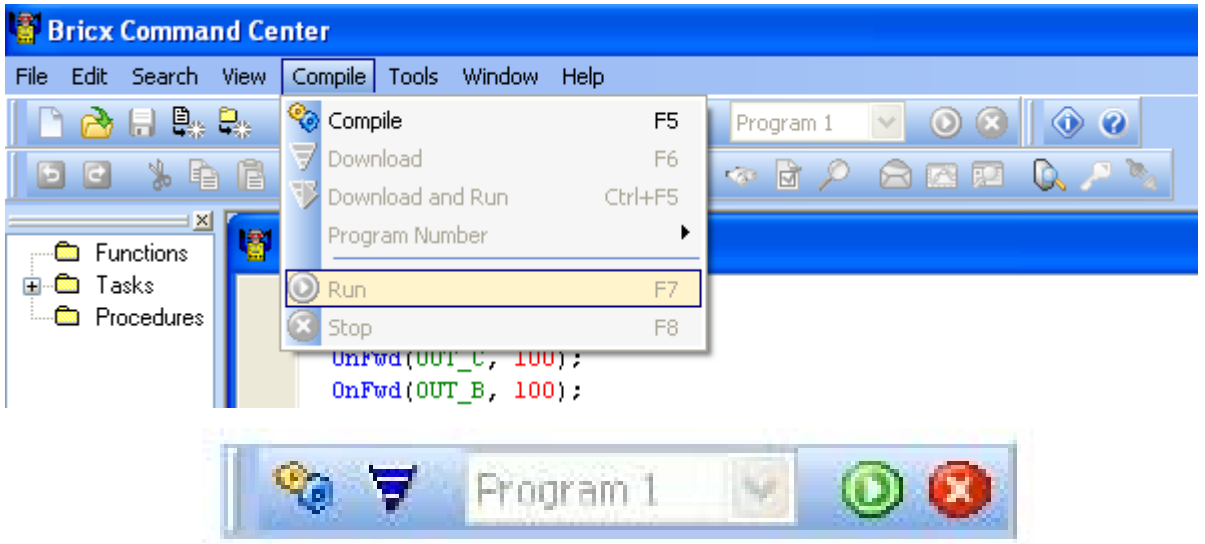
OnRev(OUT_B, 75), OnRev(OUT_C, 75) - The robot has now moved far enough so we tell it to move in reverse direction, that is, backwards. Note that we can set both motors at once using OnRev(OUT_BC) as argument. We could also have combined the first two statements this way.

Wait(400) - Again we wait for 4 seconds.

Off(OUT_BC) - And finally we switch both motors off.

That is the whole program. It moves both motors forwards for 4 seconds, then backwards for 4 seconds, and finally switches them off. You probably noticed the colors when typing in the program. They appear automatically. The colors and styles used by the editor when it performs syntax highlighting are customizable.

Running the program:
Once you have written a program, it needs to be compiled (that is, changed into binary code that the robot can understand and execute) and sent to the robot using USB cable or BT dongle (called “downloading” the program).

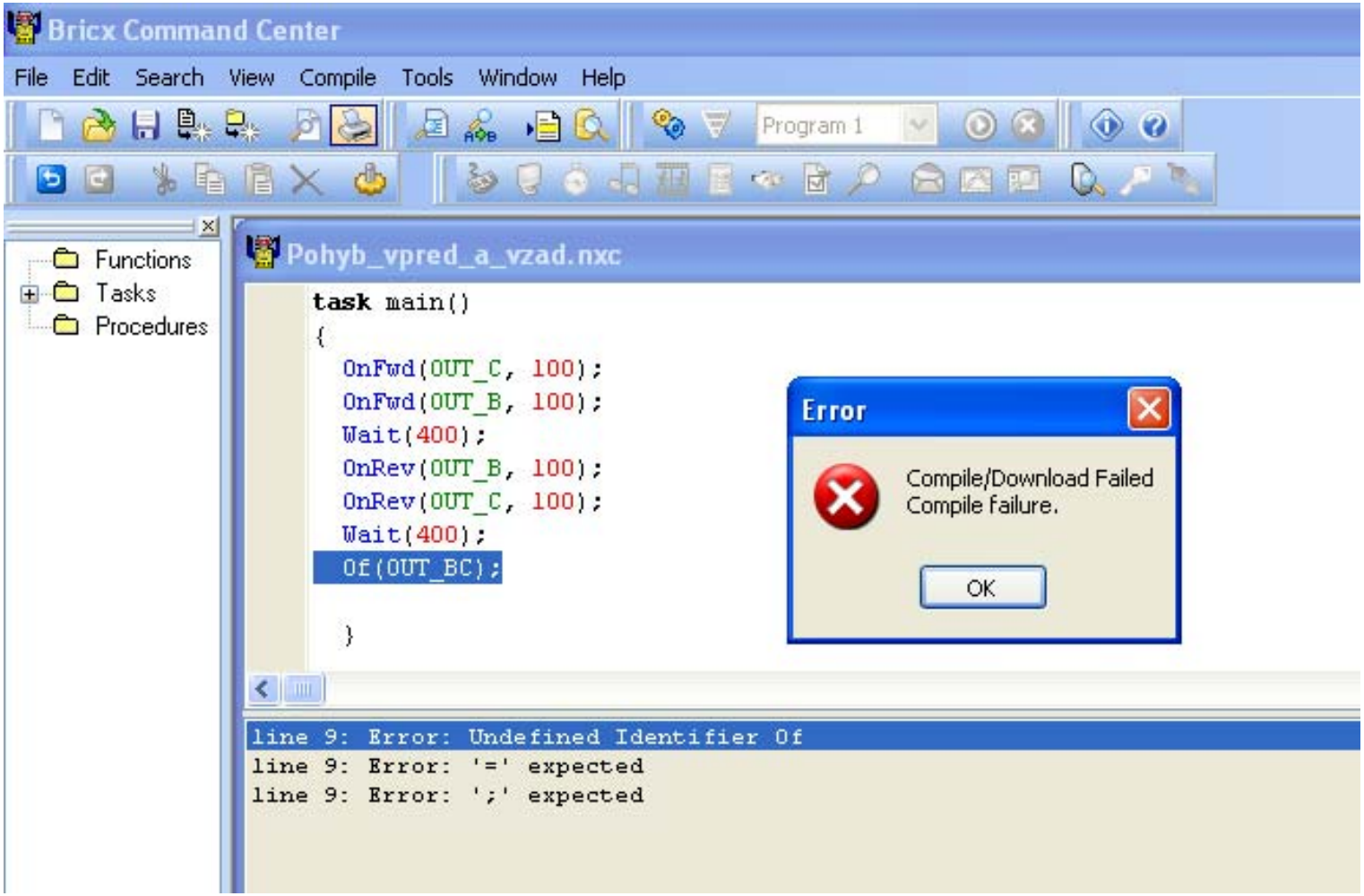


Here you can see the button that allows you to (from left to right) compile, download, run and stop the program.

Press the second button and, assuming you made no errors when typing in the program, it will be correctly compiled and downloaded. (If there are errors in your program you will be notified; see below.)

Also, to get the program run automatically, you can use the shortcut CTRL+F5 or after downloading the program you can press the green run button.

Errors in your program:
When typing in programs there is a reasonable chance that you make some errors. The compiler notices the errors and reports them to you at the bottom of the window, like in the following figure:



Define constant values , making turns, repeating commands, adding comments:

Pohyb_vpred_otoceni_opakovani_komentar.nxc

```
#define MOVE_TIME 400    // čas pro přímý pohyb
#define TURN_TIME 100    // čas pro otočení o 90 stupňů

task main()
{
  repeat (4)             // udělej čtverec
  {
  OnFwd(OUT_C, 100);     // jeď vpřed
  OnFwd(OUT_B, 100);
  Wait(MOVE_TIME);
  OnRev(OUT_C, 100);     // otoč se o 90 stupňů
  Wait(TURN_TIME);
  }
  Off(OUT_B);            // vypnutí motorů
  Off(OUT_C);
  }
```

#define The first two lines define two constants. These can now be used throughout the program. Defining constants is good for two reasons: it makes the program more readable, and it is easier to change the values. Note that BricxCC gives the define statements its own color. .

repeat() statement's parentheses indicates how many times the code inside its brackets must be repeated.

To make your program even more readable, it is good to add some comment to it. Whenever you put // on a line, the rest of that line is ignored and can be used for comments. A long comment can be put between /* and */.

Using variables:

Variables form a very important aspect of every programming language. Variables are memory locations in which we can store a value. We can use that value at different places and we can change it. Let us describe the use of variables using an example.

pohyb_po_spirale.nxc

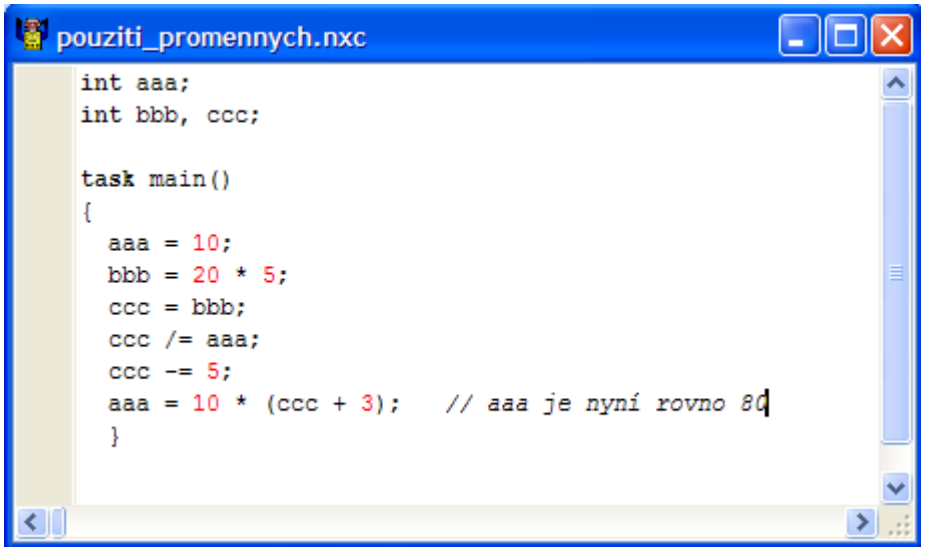
```
#define TURN_TIME 100     // čas pro otočení o 90 stupňů

int move_time;             // definujeme proměnnou

task main()
{
  move_time = 500;          // nastavíme počáteční hodnotu proměnné

  repeat (10)               // udělej čtverec
  {
  OnFwd(OUT_C, 100);       // jeď vpřed
  OnFwd(OUT_B, 100);
  Wait(move_time);
  OnRev(OUT_C, 100);       // otoč se o 90 stupňů
  Wait(TURN_TIME);
  move_time += 500;         // zvýšení hodnoty proměnné
  }
  Off(OUT_B);              // vypnutí motorů
  Off(OUT_C);
  }
```

Besides adding values to a variable we can also multiply a variable with a number using *=, subtract using -=and divide using /=. (Note that for division the result is rounded to the nearest integer.) You can also add one variable to the other, and write down more complicated expressions.



Random numbers:

In all the above programs we defined exactly what the robot was supposed to do. But things get a lot more interesting when the robot is going to do things that we don't know. We want some randomness in the motions. In NXC you can create random numbers. The following program uses this to let the robot drive around in a random way. It constantly drives forwards for a random amount of time and then makes a random turn..

The program defines two variables, and then assigns random numbers to them. **Random(600)** means a random number between 0 and 600 (the maximum value is not included in the range of numbers returned). Each time you call Random the numbers will be different.

Note that we could avoid the use of the variables by writing directly e.g. Wait(Random(600)).

Nahodny_pohyb.nxc

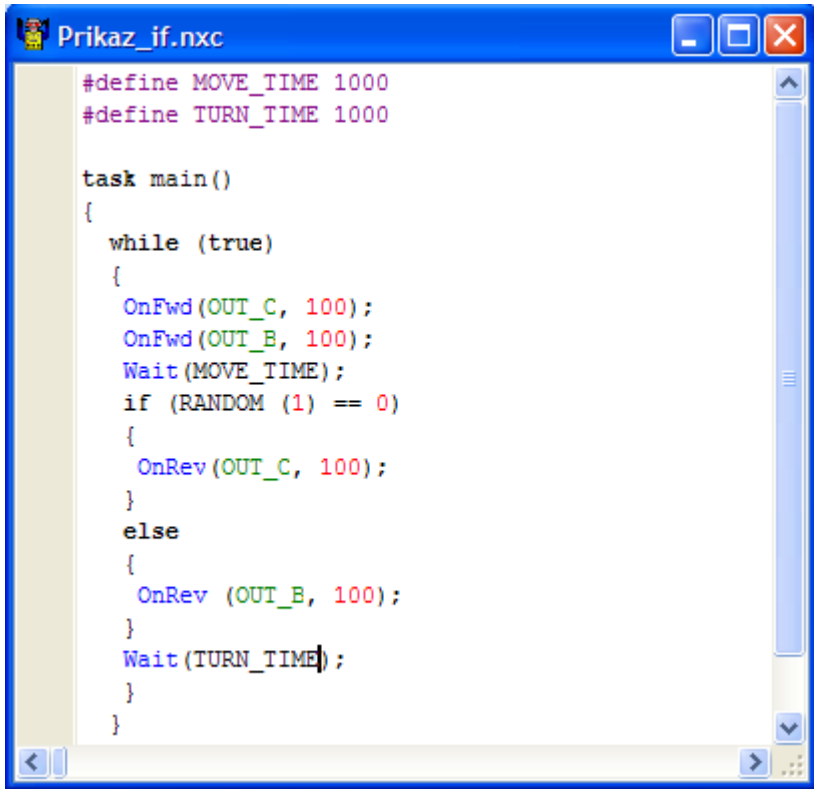
```
int move_time, turn_time;

task main()
{
  while (true)
  {
  move_time = Random (1000);
  turn_time = Random (1000);
  OnFwd(OUT_C, 100);
  OnFwd(OUT_B, 100);
  Wait(move_time);
  OnRev(OUT_C, 100);
  Wait(turn_time);
  }
  Off(OUT_B);
  Off(OUT_C);
  }
```

You also see a new type of loop here. Rather that using the repeat statement we wrote **while (true)**. The while statement repeats the statements below it as long as the condition between the parentheses is true. The special word true is always true, so the statements between the brackets are repeated forever (or at least until you press the dark grey button on NXT).

The **if** statement:

Sometimes you want that a particular part of your program is only executed in certain situations. In this case the if statement is used. Let me give an example. We will again change the program we have been working with so far, but with a new twist. We want the robot to drive along a straight line and then either make a left or a right turn. To do this we need random numbers again. We pick a random number that is either positive or negative. If the number is less than 0 we make a right turn; otherwise we make a left turn. Here is the program:



== equal to
< smaller than
<= smaller than or equal to
> larger than
>= larger than or equal to
!= not equal to

true	always true
false	never true
ttt != 3	true when ttt is not equal to 3
(ttt >= 5) && (ttt <= 10)	true when ttt lies between 5 and 10
(aaa == 10) || (bbb == 10)	true if either aaa or bbb (or both) are equal to 10

The **do** statement:

The statements between the brackets after the do part are executed as long as the condition is true. The condition has the same form as in the if statement described above. Here is an example of a program. The robot runs around randomly for 20 seconds and then stops.

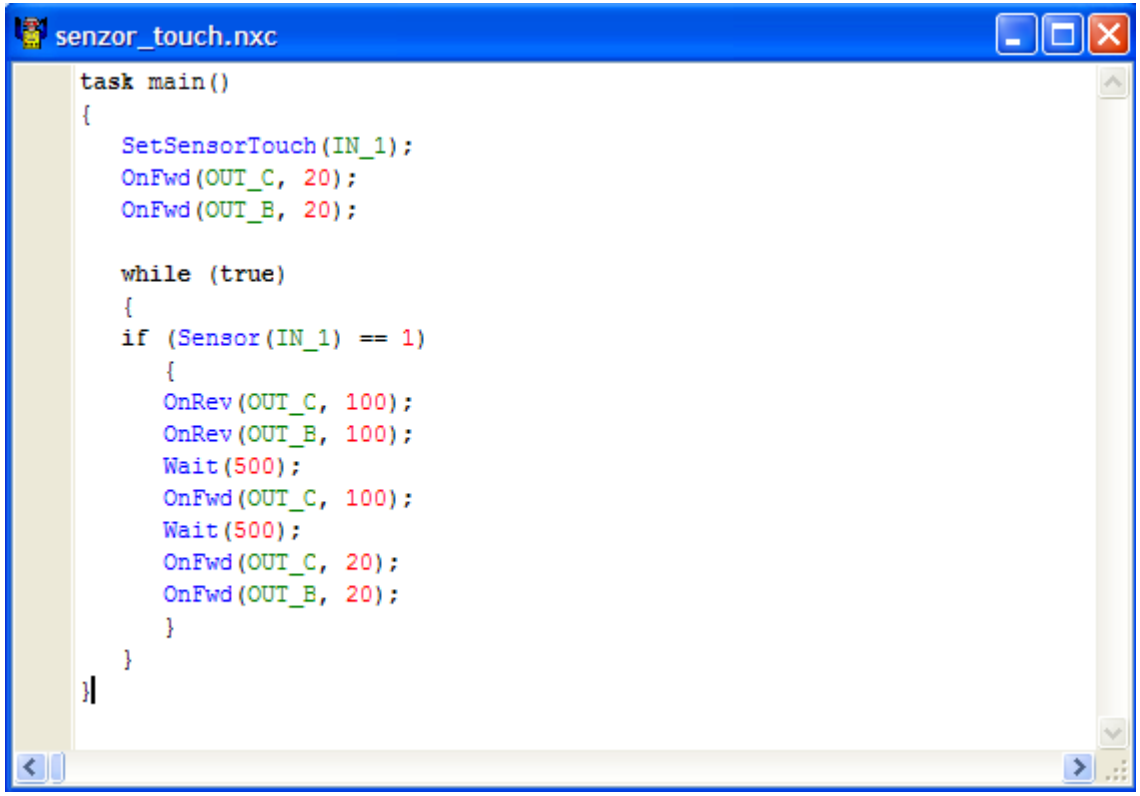
Prikaz_do.nxc

```
int move_time, turn_time, total_time;

task main()
{
  total_time = 0;
  do
  {
   move_time = Random (1000);
   turn_time = Random (1000);
   OnFwd(OUT_C, 100);
   OnFwd(OUT_B, 100);
   Wait(move_time);
   OnRev(OUT_C, 100);
   Wait(turn_time);
   total_time += move_time; total_time += turn_time;
   }
   while (total_time < 5000);
   Off (OUT_B);
   Off (OUT_C);
  }
```

Touch sensor:

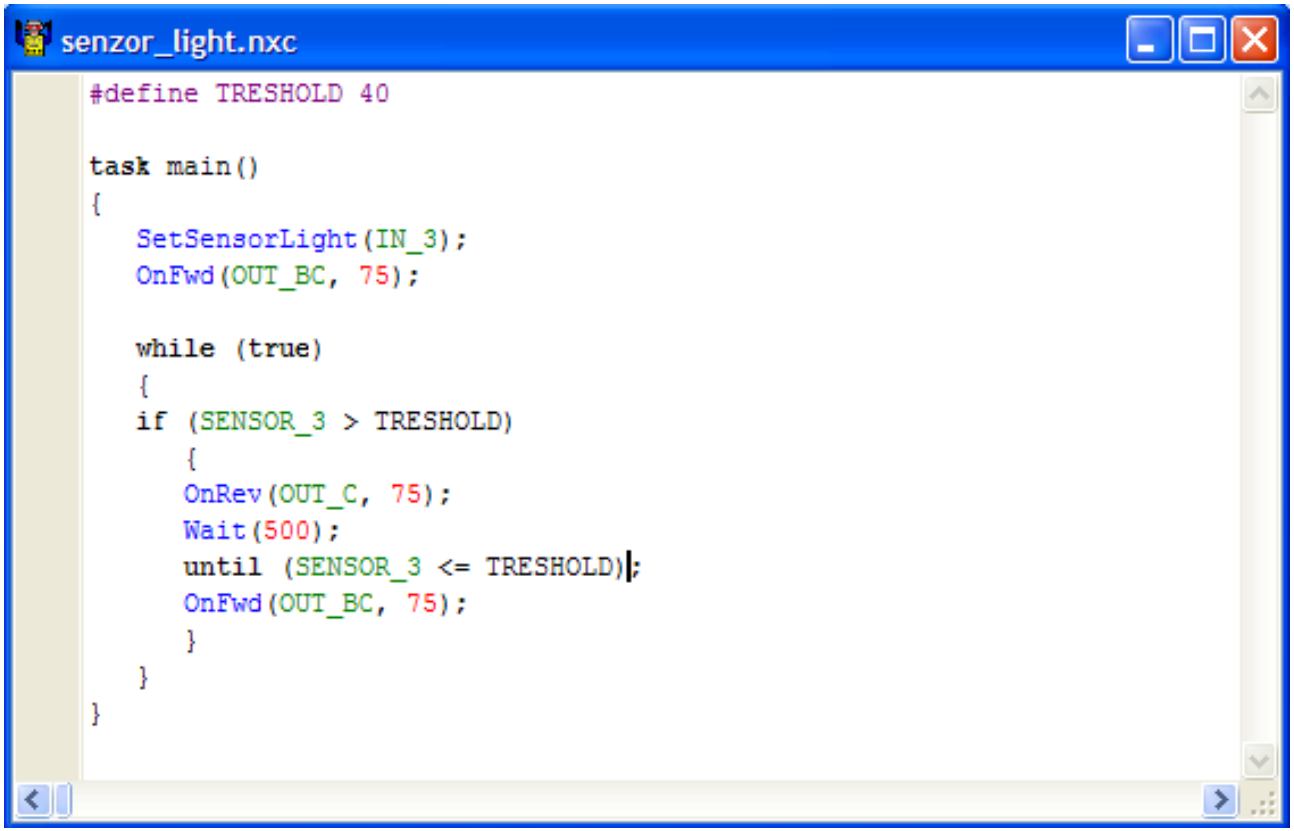
Off course you can connect sensors to NXT to make the robot react to external events.



Let us now try to make the robot avoid obstacles. Whenever the robot hits an object, we let it move back a bit, make a turn, and then continue.

Light sensor:

Besides the touch sensor, you also get a light sensor, a sound sensor and a digital ultrasonic sensor with Mindstorms NXT system. The light sensor can be triggered to emit light or not, so you can measure the amount of reflected light or ambient light in a particular direction. Measuring reflected light is particularly useful when making a robot follow a line on the floor. This is what we are going to do in the next example.



Sound sensor:

Using the sound sensor you can transform your expensive NXT set into a clapper! We are going to write a program that waits for a loud sound, and drives the robot until another sound is detected.

senzor_sound.nxc

```
#define TRESHOLD 40
#define MIC SENSOR_2

task main()
{
  SetSensorSound (IN_2);
  while (true)
  {
  until (MIC > TRESHOLD);
  OnFwd(OUT_AC, 75);
  Wait(300);
  until (MIC > TRESHOLD);
  Off(OUT_AC);
  Wait(300);
  }
}
```

Ultrasonic sensor:

Ultrasonic sensor works as a sonar: roughly speaking, it sends a burst of ultrasonic waves and measures the time needed for the waves to be reflected back by the object in sight. This is a digital sensor, meaning it has an embedded integrated device to analyze and send data. With this new sensor you can make a robot see and avoid an obstacle before actually hitting it (as is for touch sensor)..



Tasks and subroutines:

Up to now all our programs consisted of just one task. But NXC programs can have multiple tasks. It is also possible to put pieces of code in so-called subroutines that you can use in different places in your program. Using tasks and subroutines makes your programs easier to understand and more compact. In this chapter we will look at the various possibilities.

Tasks

An NXC program consists of 255 tasks at most; each of them has a unique name. The task named main must always exist, since this is the first task to be executed. The other tasks will be executed only when a running task tells them to be executed or they are explicitly scheduled in the main; main task must terminate before they can start. From that moment on, both tasks are running simultaneously. Let me show the use of tasks. We want to make a program in which the robot drives around in squares, like before. But when it hits an obstacle it should react to it. It is difficult to do this in one task, because the robot must do two things at the same moment: drive around (that is, switching motors on and off in time) and watch for sensors. So it is better to use two tasks for this, one task that moves in squares; the other that reacts to the sensors. Here is the program.

ulohy_a_podprogramy.nxc

```
mutex moveMutex;

task move_square()
{
  while (true)
  {
  Acquire(moveMutex);
  OnFwd(OUT_AC, 75); Wait(1000);
  OnRev(OUT_AC, 75); Wait(500);
  Release(moveMutex);
  }
}

task check_sensors()
{
  while (true)
  {
  if (SENSOR_1 == 1)
   {
   Acquire(moveMutex);
   OnRev(OUT_AC, 75); Wait(500);
   OnFwd(OUT_A, 75); Wait(500);
   Release(moveMutex);
   }
  }
}

task main()
{
  Precedes(move_square, check_sensors);
  SetSensorTouch(IN_1);
}
```

Subroutines

Sometimes you need the same piece of code at multiple places in your program. In this case you can put the piece of code in a subroutine and give it a name. Now you can execute this piece of code by simply calling its name from within a task. Let us look at an example.:

Podprogram.nxc

```
sub turn_around(int pwr)
{
 OnRev(OUT_C, pwr); Wait(900);
 OnFwd(OUT_AC, pwr);
}

task main()
{
  OnFwd(OUT_AC, 75);
  Wait(1000);
  turn_around(75);
  Wait(2000);
  turn_around(75);
  Wait(1000);
  turn_around(75);
  Off(OUT_AC);
}
```

Making music:

The NXT has a built-in speaker that can play tones and even sound files. This is in particular useful when you want to make the NXT tell you that something is happening. But it can also be funny to have the robot make music or talk while it runs around.

	Oktáva							
Tón	1	2	3	4	5	6	7	8
G#	52	104	208	415	831	1661	3322	
G	49	98	196	392	784	1568	3136	
F#	46	92	185	370	740	1480	2960	
F	44	87	175	349	698	1397	2794	
E	41	82	165	330	659	1319	2637	
D#	39	78	156	311	622	1245	2489	
D	37	73	147	294	587	1175	2349	
C#	35	69	139	277	554	1109	2217	
C	33	65	131	262	523	1047	2093	4186
B	31	62	123	247	494	988	1976	3951
A#	29	58	117	233	466	932	1865	3729
A	28	55	110	220	440	880	1760	3520

Tabulka 1: frekvence různých tónů

To play a tone, you can use the command PlayTone (frequency, duration)
It has two arguments. The first is the frequency in Hertz, the second the duration (in 1/1000 of a second, like in the wait command)

